Rig Routing Playbook

Putting the OCS Intelligence rigs on a private network behind a filtered school Wi-Fi, and routing them through an EC2 WireGuard relay

Open Coding Society, OCS Intelligence infrastructure

IN PROGRESS, status as of October 16, 2026

0. What is actually live right now

This build is in progress. The uplink and router are working and both rigs are on the new network, but the WireGuard tunnel to EC2 is not built yet. Anything marked **Planned** does not exist yet, so do not assume it does.

Rig	How it connects right now	Status
Rig 1 (8x GTX 1070, model serving)	**Wired.** Ethernet cable from Rig 1 into a Flint 2 LAN port.	Live
Rig 2 (8x GTX 1070, shared ML workstation)	**USB Wi-Fi dongle**, joined to the Flint 2's Wi-Fi. Temporary until it gets an Ethernet run.	Live, temporary

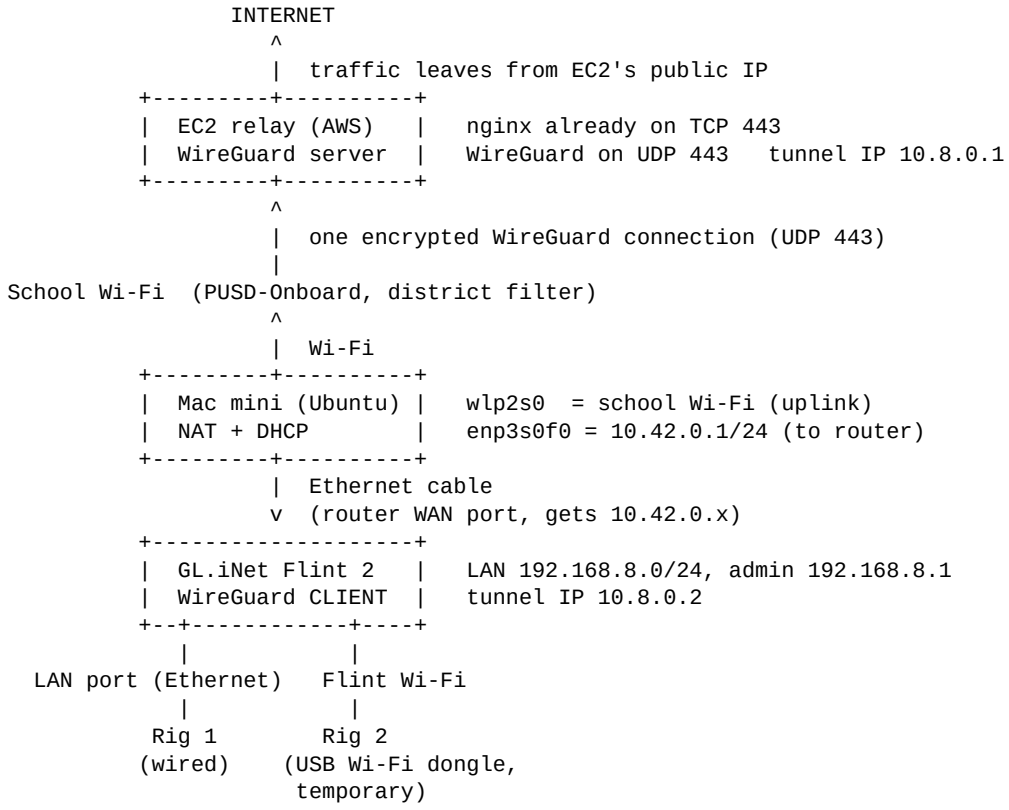
Piece	Status	Notes
Mac mini joins school Wi-Fi on its built-in card	**Live**	Proprietary `wl` driver, profile `PUSD open test`.
Mac mini shares its internet over Ethernet	**Live**	NetworkManager shared mode on `enp3s0f0`, address 10.42.0.1/24.
Router gets a WAN address from the Mac mini	**Live**	Tested with the Brume 2 (got 10.42.0.185) and then with the Flint 2.
Flint 2 broadcasts its own Wi-Fi and gives internet to clients	**Live**	Both rigs reach the internet through the whole chain.
Reboot test with the USB dongle unplugged	Pending	Must pass before anything depends on it.
UDP 443 reachability test from the Mac mini to EC2	Pending	Decides which port the tunnel uses.
WireGuard server on the EC2 relay	Planned	Steps 7 to 9 below.
Flint 2 as WireGuard client (all OCS traffic tunneled)	Planned	Steps 8 and 9 below.
EC2 can reach the rigs through the tunnel	Planned	Step 10 below.

1. The problem this solves

- The room's wall Ethernet jacks do not work, so the only uplink is the school Wi-Fi (`PUSD-Onboard`).
- The district filters outbound traffic and blocks unusual ports, so the tunnel has to use a port that gets through.
- Consumer routers cannot join the school network in a way we control, so a Linux machine (the Mac mini) joins it and hands the connection to our own router.
- Our router then owns a private OCS LAN, wired and wireless, that the school network never sees directly.
- A WireGuard tunnel from our router to the EC2 relay makes all OCS traffic leave from EC2. The school network only sees one encrypted connection to an AWS address.

Permission. The OCS team has permission for this network. If you are replicating it somewhere else, get written approval from the network owner first. Tunneling past a filter without approval is an acceptable-use violation and can get the whole uplink blocked.

2. Architecture



Why the WireGuard server is on EC2 and not on a router

- Encryption only helps until the point where the tunnel ends. If the router were the server, traffic would be decrypted inside the room and then pass through the school filter in the clear.
- A server waits for inbound connections. The router sits behind the school's network, which blocks inbound connections, so nothing outside could reach it anyway.
- A **client** connects outward, and outbound connections get through. EC2 has a public address and an open port, so EC2 is the server and the router is the client.

Why the Flint 2 and not the Brume 2

Router	Wi-Fi radio	Ports	Role here
GL.iNet Flint 2 (GL-MT6000)	Yes (Wi-Fi 6, 2.4 and 5 GHz)	1 WAN + 5 LAN	**Chosen.** Does wired LAN, Wi-Fi and the WireGuard client in one box.
GL.iNet Brume 2 (GL-MT2500A)	No	1 WAN + 1 LAN	Spare. Works as the WireGuard client only if a switch and a separate access point sit behind it.

3. Hardware, software and addresses

Hardware

Item	Details
Mac mini	Running Ubuntu with NetworkManager. Acts as the school-facing uplink.
Built-in Wi-Fi	Broadcom BCM4360, PCI ID 14e4:43a0, interface wlp2s0, needs the proprietary wl driver.
Mac mini Ethernet	Broadcom NetXtreme Gigabit, interface enp3s0f0. Caps the uplink at 1 Gb/s.
USB Wi-Fi dongle (fallback)	Ralink RT5370, USB ID 148f:5370, in-kernel rt2800usb driver. Supports OWE.
Router	GL.iNet Flint 2 (GL-MT6000). Brume 2 kept as a spare.
Switch	Optional. Plug into a Flint LAN port if you run out of ports.
Relay	An Ubuntu EC2 instance with an Elastic IP. nginx already runs on it on TCP 443.

Address plan

Every subnet must be different from every other one, or routing breaks. These are the values used here.

Network	Subnet	Who hands out addresses	Fixed addresses
School Wi-Fi	Whatever the district gives	District DHCP	None
Mac mini to router link	10.42.0.0/24	Mac mini (NetworkManager shared mode)	Mac mini = 10.42.0.1
OCS LAN (Flint wired + Wi-Fi)	192.168.8.0/24	Flint 2	Flint = 192.168.8.1, reservations for Rig 1 and Rig 2
WireGuard tunnel	10.8.0.0/24	Static, set in configs	EC2 = 10.8.0.1, Flint = 10.8.0.2

> 10.8.0.0/24 is just a common convention for WireGuard tunnels. Any private range works as long as it does not overlap the school subnet, 10.42.0.0/24 or 192.168.8.0/24.

Before you start

- SSH access to the EC2 relay with sudo, and AWS console access to edit its security group.
- The relay's Elastic IP (find it with `dig +short ai.opencodingsociety.com`).
- A keyboard and monitor (or SSH) on the Mac mini, with sudo.
- Ethernet cables: Mac mini to Flint WAN, plus one per wired device.
- About 2 hours for Steps 1 to 5, and about 1 hour for Steps 6 to 9.

Step 1: Get the Mac mini's built-in Wi-Fi working

Ubuntu does not ship a working driver for the BCM4360, so the card is invisible until you install one.

1a. Identify the real Wi-Fi hardware

1. List PCI network hardware by Broadcom's vendor ID. A plain `grep network` can miss it.

```
lspci -nn | grep -i 14e4
# expect a line ending in [14e4:43a0]  -> BCM4360
```

2. If you also have a USB dongle, compare `lsusb` with it unplugged and plugged in. Note: `0a5c:4500` is the Bluetooth controller, not Wi-Fi.

1b. Install the driver

```
sudo apt update
sudo apt install -y dkms linux-headers-$(uname -r) broadcom-sta-dkms
sudo reboot
```

1c. Verify

```
dkms status                              # broadcom-sta ... installed
lsmod | grep -E "wl|b43|bcma|ssb"        # wl loaded, b43/bcma/ssb NOT loaded
nmcli device                             # wlp2s0 is listed as wifi
nmcli device wifi list ifname wlp2s0     # school SSIDs show up
```

If Secure Boot is on, the DKMS module must be enrolled (MOK) or it will not load. A kernel update can also break the rebuild. After every kernel update, re-run the three checks above.

Step 2: Join the school Wi-Fi from the Mac mini

`PUSD-Onboard` uses **OWE** (encrypted open Wi-Fi) in transition mode, so it shows up as a pair: an OWE side and a plain open side. The `wl` driver cannot do OWE, so the built-in card joins the open side, pinned to one access point.

2a. Find the access point to pin to

```
nmcli -f SSID,BSSID,SECURITY,CHAN,SIGNAL device wifi list ifname wlp2s0 | grep -i onboard
# pick the strongest entry that is NOT marked OWE, copy its BSSID
```

2b. Create the profile

```
nmcli connection add type wifi ifname wlp2s0 con-name "PUSD open test" \
    ssid PUSD-Onboard 802-11-wireless.bssid <BSSID_FROM_2a> \
    connection.autoconnect yes
nmcli connection up "PUSD open test"
```

2c. Verify

```
ip route                         # default route must say: dev wlp2s0
curl -I https://example.com      # must return HTTP headers
```

ping 8.8.8.8 fails here and that is expected. The school network appears to drop ICMP. Test with curl, not ping.

What failed (so you do not repeat it)

Attempt	Result
Clone the dongle's OWE profile onto wlp2s0	Stuck configuring, association took too long. Deleted.
Point the original OWE profile at wlp2s0	Network could not be found (wl cannot do OWE).
OWE profile pinned to one BSSID	Network could not be found.
New open profile pinned to one BSSID	**Worked.**

Different school? If your card supports OWE (most mac80211 drivers, like the RT5370), use `wifi-sec.key-mgmt owe` instead and you get encryption. If your school uses WPA2-Enterprise (PEAP / MSCHAPv2), add the 802-1x settings with your school credentials instead.

Step 3: Share the Mac mini's internet over Ethernet

NetworkManager's **shared** mode turns on IP forwarding, NAT and a small DHCP server on the Ethernet port in one command.

1. Plug an Ethernet cable from the Mac mini into the router's **WAN** port.
2. Create and start the shared profile:

```
nmcli connection add type ethernet ifname enp3s0f0 con-name "brume-link" \
    ipv4.method shared connection.autoconnect yes
nmcli connection up "brume-link"
```

3. Verify:

```
nmcli device                                        # enp3s0f0 connected, brume-link
ip addr show enp3s0f0                               # inet 10.42.0.1/24
sysctl net.ipv4.ip_forward                          # = 1
cat /var/lib/NetworkManager/dnsmasq-enp3s0f0.leases # router's WAN lease appears here
```

The profile is named `brume-link` because it was first tested with the Brume 2. The name does not matter and it works the same with the Flint 2.

Step 4: Set up the Flint 2

1. Connect any computer to the Flint (cable to a LAN port, or its default Wi-Fi printed on the label) to reach the admin page.
2. Open `http://192.168.8.1`. Set a **strong admin password** and write it down. Anything on the OCS network can reach this page.
3. Go to **Internet**. It should already show **Ethernet** connected via DHCP with a 10.42.0.x address. If it does, leave it. Do not change the network mode.
4. Go to **Wireless**. Name the network `OCS-IoT`, use WPA2/WPA3, and set a strong password. Rig 2's dongle joins this network in Step 5.
5. From that computer, run `curl -I https://example.com`. Headers back means the whole chain works: school Wi-Fi to Mac mini to Flint.

Step 5: Connect the rigs

Rig 1: wired (current setup)

1. Run an Ethernet cable from Rig 1 into any Flint 2 LAN port.
2. In the Flint admin panel, go to **Clients**, find Rig 1 and **reserve** its address so it never changes.
3. On Rig 1, check internet with `curl -I https://example.com`.

Rig 2: USB Wi-Fi dongle (current, temporary)

Rig 2 does not have a cable run to the Flint yet, so it uses a USB Wi-Fi dongle for now.

1. Plug the dongle into Rig 2 and confirm it shows up as a `wlx...` interface:

```
nmcli device                    # look for a wifi device named wlx...
```

2. Join the Flint's Wi-Fi (replace the password):

```
nmcli device wifi connect OCS-IoT password '<FLINT_WIFI_PASSWORD>' \
    ifname $(nmcli -t -f DEVICE device | grep ^wlx)
```

3. In the Flint admin panel, reserve Rig 2's address like Rig 1.
4. On Rig 2, check internet with `curl -I https://example.com`.

Rig	Connection	Reserved IP
Rig 1	Ethernet to Flint LAN port	192.168.8.__
Rig 2	USB Wi-Fi dongle to Flint Wi-Fi (temporary)	192.168.8.__

> **Next for Rig 2:** replace the dongle with an Ethernet cable to a free Flint LAN port. A dongle is slower and less stable, which matters when Rig 2 pulls large datasets or model files.

Traffic between the two rigs stays on the Flint and never touches the school network. Only internet-bound traffic uses the Mac mini's Wi-Fi.

Step 6: Test which port the tunnel can use

WireGuard's default port is UDP 51820, which the district will very likely block. UDP 443 often gets through. Test before building anything. nginx on the relay uses **TCP** 443, so UDP 443 does not conflict with it.

1. AWS console: open the relay instance, go to **Security**, click its security group, then **Edit inbound rules**. Add: type **Custom UDP**, port **443**, source **0.0.0.0/0**. Save.
2. On the relay, start a listener:

```
sudo nc -ul 443
```

3. On the **Mac mini** (it must be the machine on the school Wi-Fi):

```
EC2_IP=$(dig +short ai.opencodingsociety.com)
echo hello | nc -u -w2 $EC2_IP 443
```

What you see on the relay	What it means	What to do
`hello` appears	UDP 443 gets out	Continue with Step 7 using port 443.
Nothing appears	UDP is blocked	Stop. Use the TCP 443 fallback in the troubleshooting table.

Press Ctrl+C on the relay to stop the listener.

Step 7: Set up the WireGuard server on the EC2 relay

SSH into the relay. Run each block one at a time and check for errors before moving on. This adds WireGuard next to the existing nginx without changing it.

7a. Install WireGuard and turn on forwarding

```
sudo apt update && sudo apt install -y wireguard
echo "net.ipv4.ip_forward=1" | sudo tee /etc/sysctl.d/99-wg.conf
sudo sysctl --system
```

7b. Make the server keys

```
sudo sh -c 'umask 077; wg genkey | tee /etc/wireguard/server_private.key \
    | wg pubkey > /etc/wireguard/server_public.key'
```

7c. Write the server config

```
IFACE=$(ip route show default | awk '{print $5}')
SERVER_PRIV=$(sudo cat /etc/wireguard/server_private.key)
sudo tee /etc/wireguard/wg0.conf >/dev/null <<EOF
[Interface]
Address = 10.8.0.1/24
ListenPort = 443
PrivateKey = $SERVER_PRIV
PostUp = iptables -A FORWARD -i wg0 -j ACCEPT
PostUp = iptables -A FORWARD -o wg0 -j ACCEPT
PostUp = iptables -t nat -A POSTROUTING -o $IFACE -j MASQUERADE
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT
PostDown = iptables -D FORWARD -o wg0 -j ACCEPT
PostDown = iptables -t nat -D POSTROUTING -o $IFACE -j MASQUERADE
EOF
sudo chmod 600 /etc/wireguard/wg0.conf
```

7d. Make the Flint's keys and register it as a peer

```
sudo sh -c 'umask 077; wg genkey | tee /etc/wireguard/flint_private.key \
    | wg pubkey > /etc/wireguard/flint_public.key'
sudo tee -a /etc/wireguard/wg0.conf >/dev/null <<EOF

[Peer]
PublicKey = $(sudo cat /etc/wireguard/flint_public.key)
AllowedIPs = 10.8.0.2/32
EOF
```

7e. Start it and make it survive reboots

```
sudo systemctl enable --now wg-quick@wg0
sudo wg show                    # interface wg0, listening port 443
sudo ufw allow 443/udp          # only if ufw is enabled
```

7f. Print the Flint's config

```
EC2_IP=$(curl -s https://checkip.amazonaws.com)
cat <<EOF
[Interface]
PrivateKey = $(sudo cat /etc/wireguard/flint_private.key)
Address = 10.8.0.2/32
DNS = 1.1.1.1
MTU = 1380

[Peer]
PublicKey = $(sudo cat /etc/wireguard/server_public.key)
Endpoint = $EC2_IP:443
AllowedIPs = 0.0.0.0/0
PersistentKeepalive = 25
EOF
```

Save the output on your laptop as `flint.conf`.

> **flint.conf contains a private key.** Never paste it into GitHub, Slack, a shared doc or a chat. If it leaks, delete the peer on EC2 and redo 7d and 7f.

What each setting does

Setting	Meaning
ListenPort = 443	The relay listens for WireGuard on UDP 443, the port that passed Step 6.
PostUp MASQUERADE	Rewrites tunnel traffic so it leaves to the internet from EC2's public IP.
AllowedIPs = 0.0.0.0/0 (Flint)	Send all of the Flint's internet traffic into the tunnel (full tunnel).
PersistentKeepalive = 25	Sends a small packet every 25 s so the school's NAT keeps the connection open.
MTU = 1380	Leaves room for tunnel overhead so large packets do not get dropped.

Step 8: Add the WireGuard client to the Flint 2

Menu names can differ a little between firmware versions.

1. Open `http://192.168.8.1` from a device on the OCS network.
2. Go to **VPN**, then **WireGuard Client**.
3. Choose **Add a New WireGuard Configuration**, then **Upload** (or paste) `flint.conf`. Name it `ocs-ec2` and save.

4. Turn the toggle on to connect.
5. In the VPN options, choose **Global Proxy** so every device on the OCS network uses the tunnel.
6. Turn on **Block Non-VPN Traffic** (the kill switch) so nothing leaks out through the school filter if the tunnel drops.

With the kill switch on, **if the tunnel goes down the whole OCS network loses internet**, though local traffic between devices keeps working. That is intended. Check Step 9 first whenever internet stops.

Step 9: Test the tunnel end to end

#	Where	Command or action	Pass looks like
1	EC2 relay	`sudo wg show`	Peer 10.8.0.2 has a recent **latest handshake** and transfer counters going up.
2	Rig 2 (dongle)	`curl -s https://ifconfig.me`	Shows the **EC2 Elastic IP**, not the school's address.
3	Rig 1 (wired)	`curl -s https://checkip.amazonaws.com`	Same EC2 IP.
4	Any device	Open a site the district normally filters	It loads.
5	Flint admin	Turn the VPN off for a moment	Clients lose internet (kill switch works). Turn it back on.
6	Mac mini	`sudo reboot`, wait, retest 1 to 3	Everything returns with no manual steps.

Step 10 (planned): Reach the rigs from EC2 through the tunnel

Not done yet. After Step 9 passes, these steps let the relay talk to each rig directly over the tunnel.

1. On EC2, change the Flint peer's line in `/etc/wireguard/wg0.conf` to `AllowedIPs = 10.8.0.2/32, 192.168.8.0/24`, then `sudo systemctl restart wg-quick@wg0`.
2. On the Flint, in the WireGuard client settings, turn on **remote access to LAN**.
3. From EC2, ping or curl each rig on its reserved IP, for example `curl http://192.168.8.<rig>:<port>/`. Both Rig 1 and Rig 2 should answer.

11. Troubleshooting

Symptom	Likely cause	Fix
wlp2s0 missing from `nmcli device`	wl driver not loaded (kernel update or Secure Boot)	Re-run Step 1c. Reinstall broadcom-sta-dkms. Enroll MOK if Secure Boot is on. Use the USB dongle meanwhile.
PUSD open test will not connect	The pinned access point is down or you moved rooms	Redo Step 2a and update the BSSID with `nmcli connection modify`.
Router has no WAN address	Cable in a LAN port, or brume-link is down	Use the WAN port. Run `nmcli connection up brume-link`.
ping fails but browsing works	School drops ICMP	Normal. Test with curl.
No handshake in `wg show`	UDP 443 blocked, security group missing, or wrong Endpoint IP	Recheck Step 6 and the inbound rule. Confirm the Endpoint is the Elastic IP.
Handshake works but some sites hang	MTU too high	Set MTU = 1280 in flint.conf and re-upload it.
UDP is blocked entirely	District filters all UDP out	Fall back to OpenVPN over TCP 443 (the Flint supports it). nginx already owns TCP 443 on the relay, so it needs port sharing or a second address.
Everything lost internet	Tunnel down plus kill switch	Check `sudo wg show` on EC2 and the VPN page on the Flint.

12. Known risks

- **Unencrypted uplink.** The built-in card is on the open side of the school Wi-Fi. Until the tunnel is live, anything not using HTTPS can be seen by others on the school Wi-Fi.
- **Throughput.** Every byte to the internet goes over one Wi-Fi link, the Mac mini and then EC2. Measure with `iperf3` before moving large datasets or model files over it.
- **Double NAT.** The Mac mini and the Flint both NAT. Outbound is fine. Inbound access should come through the tunnel, not port forwarding.
- **Single points of failure.** One Mac mini, one access point pin and one EC2 instance carry the whole network.
- **Driver fragility.** wl is out of tree and unmaintained. A routine kernel update can take down the uplink.
- **EC2 becomes the trust boundary** for both rigs. Keep its security group tight and its keys private.

13. Handy commands

```
# Mac mini
ip route                                  # which interface has the default route
nmcli device                              # adapter and profile status
nmcli connection show                     # saved profiles
sysctl net.ipv4.ip_forward                # must be 1
cat /var/lib/NetworkManager/dnsmasq-enp3s0f0.leases

# Fallback: use the USB dongle's encrypted OWE profile
nmcli connection up "PUSD onboard" ifname $(nmcli -t -f DEVICE device | grep ^wlx)

# EC2 relay
sudo wg show
sudo systemctl status wg-quick@wg0
sudo systemctl restart wg-quick@wg0
sudo journalctl -u wg-quick@wg0 -n 50
```

14. Glossary

Term	Meaning
Uplink	The connection a network uses to reach the internet. Here, the Mac mini's school Wi-Fi.
WAN / LAN port	WAN is the router's internet side. LAN ports are for your own devices.
NAT	Lets many devices share one address by rewriting addresses on the way out.
DHCP	Hands out IP addresses automatically. A reservation makes one device always get the same address.
OWE	Opportunistic Wireless Encryption. Encrypts open Wi-Fi without a password.
BSSID	The hardware address of one specific access point broadcasting a network name.
WireGuard	A fast, simple VPN. One side is the server (waits), the other is the client (connects out).
Full tunnel	All internet traffic goes through the VPN, set by AllowedIPs = 0.0.0.0/0.
Kill switch	Blocks traffic if the VPN drops, so nothing leaks around it.
Relay	The EC2 server with a public IP that sits outside the district network.